

Python Programming — Detailed Notes (Beginner Level)

Unit 1 & Unit 2 (with Definitions and Examples)

UNIT 1: INTRODUCTION

1.1 Variables

Definition: A variable is a named location in memory used to store a value that can change during program execution. Python is dynamically typed, meaning you don't declare a type — it is inferred automatically from the assigned value.

```
x = 10          # x refers to an integer
name = "Riya"   # name refers to a string
price = 99.5     # price refers to a float
```

Rules for naming variables:

- Must start with a letter or underscore (not a digit)
- Can contain letters, digits, underscores
- Case-sensitive (`age` and `Age` are different variables)
- Cannot use reserved keywords (`if` , `for` , `class` , etc.)

1.2 Expressions

Definition: An expression is any combination of values, variables, and operators that Python can evaluate to produce a single value.

```
3 + 5           # expression → evaluates to 8
x * 2           # expression using a variable
len("hello")    # expression using a function call
```

1.3 Statements

Definition: A statement is a complete instruction that Python can execute. It performs an action, such as creating a variable, printing output, or making a decision — as opposed to an expression, which just produces a value.

```
x = 10          # assignment statement
print(x)        # function-call statement
if x > 5:       # conditional statement
    print("big")
```

1.4 Operators and Types of Operators

Definition: An operator is a symbol that performs an operation on one or more values (called operands).

Type	Definition	Operators	Example	Result
Arithmetic	performs mathematical calculations	+ - * / // % **	7 // 2	3
Relational (Comparison)	compares two values, returns True/False	== != > < >= <=	5 > 3	True
Logical	combines conditional statements	and or not	True and False	False
Assignment	assigns/updates a variable's value	= += -= *= /=	x += 1	increments x
Bitwise	operates on the binary representation	& ^ ~ << >>	5 & 3	1
Membership	tests whether a value exists in a sequence	in, not in	'a' in 'cat'	True
Identity	tests whether two variables refer to the same object	is, is not	x is None	checks same object

```
a = 10
b = 3
print(a + b)    # 13
print(a / b)    # 3.333... (true division – always returns a float)
print(a // b)   # 3 (floor division – rounds down to nearest whole number)
print(a % b)    # 1 (modulus – remainder after division)
print(a ** b)   # 1000 (exponentiation)
```

1.5 Order of Operations (Operator Precedence)

Definition: Operator precedence is the fixed order in which Python evaluates operators within an expression when multiple operators appear together, similar to the "BODMAS" rule in mathematics.

Order (highest to lowest):

1. `()` Parentheses
2. `**` Exponentiation
3. `+x`, `-x`, `~x` Unary plus/minus
4. `*` `/` `//` `%` Multiplication/Division/Modulus
5. `+` `-` Addition/Subtraction
6. Comparison operators (`==` `!=` `>` `<` `...`)
7. `not`
8. `and`
9. `or`

```
result = 2 + 3 * 4      # multiplication happens first → 2 + 12 = 14
result2 = (2 + 3) * 4   # parentheses force addition first → 5 * 4 = 20
```

1.6 Data Types

Definition: A data type defines the kind of value a variable holds and what operations can be performed on it.

Type	Example	Description
int	10	whole numbers
float	10.5	decimal numbers
str	"hello"	text, a sequence of characters
bool	True / False	logical/boolean value
complex	3+4j	complex numbers
list	[1,2,3]	ordered, mutable collection
tuple	(1,2,3)	ordered, immutable collection
dict	{"a":1}	key-value pairs
set	{1,2,3}	unordered collection of unique items

```
x = 5
print(type(x))      # <class 'int'>
```

Type conversion (casting): converting a value from one data type to another.

```
a = "10"
b = int(a)          # string → int → 10
c = str(25)         # int → string → "25"
d = float("3.14")   # string → float → 3.14
```

1.7 Asking the User for Input

Definition: The `input()` function pauses the program and waits for the user to type something on the keyboard; it always returns the result as a **string**, regardless of what the user typed.

```
name = input("Enter your name: ")
print("Hello,", name)

age = int(input("Enter your age: "))  # must convert to int before doing math
print("Next year you'll be", age + 1)
```

⚠ Common beginner mistake:

```
num1 = input("Enter a number: ")
num2 = input("Enter another number: ")
print(num1 + num2)           # WRONG: "5"+"3" = "53" (string concatenation)
print(int(num1) + int(num2)) # CORRECT: 8
```

1.8 Conditional Execution

Definition: Conditional execution allows a program to make decisions and run different blocks of code depending on whether a condition is `True` or `False`.

if statement

Definition: Executes a block of code only if the given condition is `True`.

```
age = 20
if age >= 18:
    print("You are an adult")
```

if-else statement

Definition: Executes one block if the condition is `True`, and a different block if it's `False`.

```
num = 7
if num % 2 == 0:
    print("Even")
else:
    print("Odd")
```

nested if

Definition: An `if` (or `if-else`) statement placed inside another `if` (or `else`) block, used to check a condition only after an outer condition is satisfied.

```
num = 15
if num > 0:
    if num % 2 == 0:
        print("Positive Even")
    else:
        print("Positive Odd")
else:
```

```
print("Not positive")
```

elif (else-if ladder)

Definition: Short for "else if" — allows checking multiple conditions in sequence, stopping at the first one that is True.

```
marks = 72
if marks >= 90:
    print("Grade A")
elif marks >= 75:
    print("Grade B")
elif marks >= 60:
    print("Grade C")
else:
    print("Fail")
```

Shorthand if (ternary / conditional expression)

Definition: A one-line way of writing a simple if-else, useful for assigning a value based on a condition.

```
a = 10
b = 20
print("A is greater") if a > b else print("B is greater")

max_val = a if a > b else b    # common ternary pattern
print(max_val)
```

1.9 Iterative Statements

Definition: Iterative statements (loops) repeat a block of code multiple times, either while a condition holds or over each item in a sequence.

while loop

Definition: Repeats a block of code as long as a given condition remains True. Used when the number of repetitions isn't known in advance.

```
i = 1
while i <= 5:
    print(i)
```

```
a=int(input("enter ur number"))
if(a>0):
    if(a>10):
        print("Positive number is greater than 10")
    else:
        print("Positive number is less than 10")
else:
    print("negative number")
```

```
i += 1
```

for loop

Definition: Repeats a block of code once for each item in a sequence (such as a string, list, or range).

```
for letter in "Python":  
    print(letter)  
  
fruits = ["apple", "banana", "cherry"]  
for fruit in fruits:  
    print(fruit)
```

range() function

Definition: Generates a sequence of numbers, commonly used to control the number of iterations in a `for` loop. **Syntax:** `range(start, stop, step)` — `stop` is excluded.

```
range(5)          # 0,1,2,3,4  
range(2, 8)       # 2,3,4,5,6,7  
range(0, 10, 2)   # 0,2,4,6,8 (step = 2)  
  
for i in range(1, 6):  
    print(i)      # prints 1 to 5
```

Loop control keywords:

- `break` — **Definition:** immediately exits the loop entirely.
- `continue` — **Definition:** skips the rest of the current iteration and moves to the next one.

```
for i in range(10):  
    if i == 5:  
        break          # stops the loop completely when i is 5  
    if i % 2 == 0:  
        continue      # skips printing even numbers  
    print(i)
```

UNIT 2: FUNCTIONS AND STRINGS

2.1 Functions

Definition: A function is a named, reusable block of code that performs a specific task, defined using the `def` keyword, which can optionally take input and return output.

```
def greet(name):  
    print("Hello,", name)  
  
greet("Aman")      # calling the function
```

2.2 Types of Functions

Definition: Python functions can be classified by where they come from and how they behave.

1. **Built-in functions — Definition:** functions already provided by Python, ready to use without definition (`print()` , `len()` , `input()` , `type()`).
2. **User-defined functions — Definition:** functions created by the programmer using `def` to perform custom tasks.
3. **Lambda (anonymous) functions — Definition:** small, unnamed, one-line functions defined with the `lambda` keyword.
4. **Recursive functions — Definition:** a function that calls itself to solve a smaller instance of the same problem.

2.3 Parameters and Arguments

Definition: A **parameter** is a variable listed inside the parentheses in a function definition; an **argument** is the actual value passed to the function when it is called.

```
def add(a, b):          # a, b are parameters  
    return a + b  
  
result = add(5, 3)      # 5, 3 are arguments  
print(result)           # 8
```

Types of arguments:

```
# Default argument — used if no value is passed for that parameter  
def greet(name="Guest"):  
    print("Hello", name)
```

```

greet()                # Hello Guest
greet("Riya")          # Hello Riya

# Keyword argument – passed using the parameter's name, so order doesn't matter
def student(name, age):
    print(name, age)
student(age=20, name="Aman")

# Variable-length argument – accepts any number of arguments, collected into a tuple
def total(*numbers):
    return sum(numbers)
print(total(1, 2, 3, 4))    # 10

```

2.4 Lambda Functions

Definition: A lambda function is a small, anonymous (unnamed) function that can take any number of arguments but contains only a single expression, whose result is automatically returned. Syntax: `lambda arguments: expression`.

```

square = lambda x: x * x
print(square(5))          # 25

add = lambda a, b: a + b
print(add(3, 4))          # 7

# Commonly used with map(), filter(), sorted()
nums = [1, 2, 3, 4, 5]
squares = list(map(lambda x: x**2, nums))
print(squares)             # [1, 4, 9, 16, 25]

evens = list(filter(lambda x: x % 2 == 0, nums))
print(evens)               # [2, 4]

```

2.5 Recursive Functions

Definition: A recursive function is a function that calls itself, breaking a large problem into smaller, identical sub-problems, until it reaches a **base case** — a condition that stops further recursive calls.

```

def factorial(n):
    if n == 0:                # base case – stops the recursion
        return 1
    else:

```

```

        return n * factorial(n - 1)    # recursive call

print(factorial(5))                    # 120

def fibonacci(n):
    if n <= 1:                        # base case
        return n
    return fibonacci(n-1) + fibonacci(n-2)    # recursive calls

for i in range(7):
    print(fibonacci(i), end=" ")    # 0 1 1 2 3 5 8

```

2.6 Strings

Definition: A string is a sequence of characters enclosed in single, double, or triple quotes, used to represent text.

Strings are immutable

Definition: Immutable means that once a string is created, its individual characters cannot be changed, added, or removed in place — any "modification" actually creates a brand-new string.

```

s = "Hello"
# s[0] = "J"    # ❌ Error: strings don't support item assignment
s = "J" + s[1:] # ✅ creates a new string "Jello"
print(s)

```

Traversal through a string with a loop

Definition: Traversal means visiting each character of the string one by one, typically using a loop.

```

word = "Python"
for ch in word:
    print(ch)

# Using index
for i in range(len(word)):
    print(i, word[i])

```

String slices

Definition: A slice is a way to extract a portion (substring) of a string using the syntax `string[start:stop:step]` , where `stop` is excluded.

```
s = "Programming"
print(s[0:6])      # 'Progra'
print(s[:4])       # 'Prog'   (from the start)
print(s[4:])       # 'ramming' (to the end)
print(s[-4:])      # 'ming'   (negative index counts from the end)
print(s[::2])      # 'Pormig' (every 2nd character)
print(s[::-1])     # 'gnimmargorP' (reversed string)
```

String functions and methods

Definition: String methods are built-in functions that operate on string objects to search, transform, or format text, called using dot notation (`string.method()`).

```
s = "  Hello World  "

len(s)                # length of the string: 15
s.strip()             # removes leading/trailing whitespace → "Hello World"
s.lower()             # converts to lowercase
s.upper()             # converts to uppercase
s.replace("World", "Python") # replaces a substring
s.split()             # splits into a list of words → ['Hello', 'World']
"-".join(["a","b","c"]) # joins a list into a string using a separator → "a-b-c"
s.find("World")       # returns the index of first occurrence, or -1 if not found
s.count("l")          # counts how many times a substring occurs
s.startswith("  He")  # checks if the string starts with given text
s.endswith("d  ")     # checks if the string ends with given text
s.isdigit()           # True if all characters are digits
s.isalpha()           # True if all characters are letters

name = "python"
print(name.capitalize()) # "Python"
print(f"My name is {name}") # f-string: embeds variables directly into a string
```

Quick Practice Ideas (for students)

1. Write a program to check if a number is prime (uses `if` , `for` , `%`).
2. Write a function `is_palindrome(s)` that checks if a string reads the same backward

(uses slicing `s[::-1]`).

3. Write a recursive function to compute the sum of digits of a number.
4. Use a `lambda` with `sorted()` to sort a list of tuples by the second element.